

Architectural Blueprints

OmniERP

Blueprint I — Phase 8: The AI Subsystem

Blueprint II — Phase 0: The Refactoring Execution Plan

No compromise — Tier A + Tier B — refactor in place, no rewrite

DOCUMENT	Architectural Blueprints I & II
VERSION	1.0
DATE	26 July 2026
AUTHOR	Principal Architecture & Product
AUDIENCE	Engineering — execution reference
STATUS	Execution-ready

BLUEPRINT I — Phase 8: The AI Subsystem

I.0 The Thesis

Every ERP vendor will have access to the same models. Odoo will ship a copilot. Zoho already has one. Daftra will bolt GPT onto a search box within the year. **Model access is not a moat and will never be one.**

What differs is the substrate. An AI system's ceiling is set by the structure of the data it reasons over, and this is where the fourteen months of "invincible core" work converts into an unfair advantage:

ASK AN AI...	LEGACY ERP CAN ANSWER	OMNIERP CAN ANSWER	BECAUSE
"What was my revenue last month?"	✓	✓	Trivial. Everyone wins
"What was my <i>true</i> margin on this SKU?"	✗ Approximate	✓ Exact	COGS captured at time of sale (FR-427), not recomputed
"Why did my margin drop 4 points?"	✗	✓ Decomposed	Cost layers let you separate price effect from cost effect from mix effect
"Which customer is costing me money?"	✗	✓	COD settlement reconciliation gives true per-order profit after carrier fees
"Where did this defective batch go?"	✗	✓ In 3 seconds	Bidirectional batch traceability (FR-355)
"Will I have cash on the 15th?"	✗	✓	AR aging + AP schedule + carrier receivables + post-dated cheques + LC commitments, all structured
"Is this new customer going to return the order?"	✗ Impossible	✓	Trust Network. No competitor has this data at any price
"Is someone stealing from me?"	✗	✓	Hash-chained ledger + immutable audit + approval trail + four-eyes records
"Am I doing badly, or is the whole market?"	✗	✓	Cross-tenant anonymized benchmarking

The strategic statement: *Legacy ERPs will add AI to unstructured data and produce plausible-sounding guesses. We will add AI to a hash-chained, dimensionally-complete, cost-layered, network-connected ledger and produce provable answers.* The gap is not in the model. It is in the fourteen months.

Corollary that should shape Phase 8 planning: the AI work is *small* if the core is right. That is why FR-1501 (the reporting service layer) matters so much — build the ~60 typed report functions during Phases 1–5 because you need them for screens anyway, and Phase 8 becomes an adapter, not a subsystem.

I.1 The Copilot — From Query Box to Chief of Staff

Level 1 — Grounded Retrieval (weeks 1–2 of Phase 8)

The model's only job is to select a typed function from the reporting service layer and fill its parameters. The function executes real, indexed, permission-checked SQL. **The model never sees a database and never writes a query.**

```
"ليه صافي ربحي من موبايلات سامسونج الشهر ده بعد المرتجعات؟"
↓ intent classification (local Qwen 2.5 7B, on-soil GPU)
↓ get_item_profitability(
  period=current_month,
  category="Samsung Mobiles",
  net_of_returns=True,
  include_cod_fees=True
)
↓ real SQL over cost_consumptions + invoice lines + carrier settlements
↓ rendered as a number card + margin waterfall + "View 47 transactions"
```

Answers render as **components, not prose** — a figure, a chart, a table, always with a drill-through. An unverifiable AI answer about money is worse than no answer.

Level 2 — Multi-Hop Reasoning (weeks 3-4)

The step legacy systems cannot take, because it requires *chaining* across dimensions that only exist if the core is dimensionally complete.

"ليه مكسبي قل الشهر ده؟" (*Why did my profit drop this month?*)

The Copilot runs a **variance decomposition** — a deterministic analytical routine the model orchestrates rather than invents:

```
Gross margin: 31.2% → 26.8%   (-4.4 pts)

Decomposition:
  Cost effect      -2.9 pts   Landed cost up 18% on 3 imported SKUs
                               (EGP devaluation, shipment #IMP-0912)
  Mix effect       -1.1 pts   Sales shifted toward low-margin accessories
  Price effect     -0.9 pts   Average discount up from 4.1% → 7.3%
                               (driven by 2 salespeople — see breakdown)
  Return effect    +0.5 pts   Return rate down 2.1pts (Trust Network:
                               41 high-risk orders declined, ~EGP 6,400 saved)
                               _____
                               -4.4 pts
```

Every one of those four lines requires a data structure a legacy ERP does not have. Cost effect needs cost layers with landed cost lineage. Mix effect needs per-line COGS at time of sale. Price effect needs discount tracking with salesperson attribution. Return effect needs COD settlement plus Trust Network decline logging.

Further multi-hop examples the architecture unlocks: - "ليه الكاش قليل والمبيعات كويسة؟" → traces to Cash-with-Carrier aging: Bosta owes EGP 340k, 22 days outstanding, above their normal 9-day cycle - "إيه أسوأ عميل عندي؟" → not by revenue, but by profit after returns, after COD fees, after credit days consumed - "الشحنة دي كسبانة ولا خسرانة؟" → full landed-cost-to-sell-through analysis per import shipment

Level 3 — Proactive Intelligence (weeks 5-6)

The Copilot stops waiting to be asked. A **morning brief**, generated overnight, delivered in-app and to WhatsApp:

صباح الخير يوسف - ملخص النهارده

⚠️ حاجات محتاجة قرار 3

- فاتورة مورد بره حدود المطابقة (فرق 12%) - محتاجة موافقتك
- رصيد "جراب سيليكون شفاف" هيخلص خلال 4 أيام - مدة التوريد 11 يوم
- فاصلهم 14 ساعة على مهلة الـ 72 ساعة B2C | إيصالات 6

💰 الفلوس

- متوقع تستلم 84,200 ج.م من بوسطة يوم الأحد
- عليك 61,000 ج.م لموردين خلال 6 أيام
- ✅ السيولة المتوقعة يوم 15: 143,000 ج.م

📊 ملاحظة

- هامش ربح فئة "شواحن" نزل 6 نقاط الأسبوع ده
- %وصلت بتكلفة أعلى 22 IMP-0931 السبب: شحنة

Design constraint (AI-B01): the brief is capped at what fits on one phone screen without scrolling, and every item must be either an action or a genuine anomaly. A brief that becomes routine noise is uninstalled within a week. Rank by estimated EGP impact and show the top N only.

Level 4 — Agentic Execution (weeks 7–8)

Here is the elegant part, and it is a direct consequence of building approval workflows first.

The hardest problem with write-enabled AI agents in a financial system is the safety rail: how do you let an AI create documents without letting it corrupt the books? Most vendors answer this by bolting on a bespoke confirmation dialog and hoping.

You already built the answer in Phase 1d. An AI-originated document enters at `draft`, and if any approval rule matches it goes to `pending_approval` exactly like a human-created one. The AI **cannot be an approver** — FR-1206's segregation of duties is a database-level constraint, and "the AI" is simply never a valid approver identity. FR-1212's version hash means an AI-drafted document that is later altered voids its approval. FR-1211's immutable audit records `source='ai_assisted'` alongside the model version and the prompt that produced it.

The approval engine is the AI safety layer, and it required zero additional work. This is what "the core enables the AI" means concretely.

Agentic capabilities this unlocks safely: - "اعمل لي أوامر شراء لكل الأصناف اللي قريت تخلص" → drafts 7 POs across 4 suppliers, grouped, each routed through the normal approval chain - "ابعت تنبيه للعملاء المتأخرين أكثر من 45 يوم" → drafts WhatsApp messages, queued for review - "طبق زيادة 8% على أسعار فئة الإكسسوارات" → drafts a price change, which hits the FR-1217 four-eyes rule automatically - "صالح" → runs the settlement match, posts what matches cleanly, routes exceptions to a human queue

Level 5 — The Advisor (Phase 8c)

Trained on the merchant's own history plus anonymized network patterns:

"You've reordered 400 units of this SKU. Over the last 3 Ramadans, this category's demand dropped 40% in the second half of the month. The network shows 61 similar merchants reduced orders by an average of 30% at this point. Consider 250 units — it frees roughly EGP 42,000 of working capital, and your supplier's lead time of 11 days lets you top up if demand holds."

That is not a chatbot. That is a competent operations manager, and it is only possible because of the cross-tenant substrate.

I.2 Document Intelligence — Beyond OCR

Vendor-bill OCR is table stakes; every competitor will ship it. The differentiated play is the documents that are *hard*, *Arabic*, and *tied to a data structure only you have*.

The Killer Application: Customs Clearance Documents

For the importer persona, the single most painful, error-prone, expensive-to-get-wrong task in the business is turning a stack of clearance paperwork into accurate landed cost. It takes a clerk a full day per shipment and it is usually wrong.

Merchant photographs 6 documents:
البيان الجمركي (customs declaration)
Commercial invoice
Bill of lading
Packing list
Clearance agent invoice
Bank LC settlement advice

- ↓ VLM extraction (Qwen2.5-VL class, on-soil)
- ↓ cross-document validation:
 - HS codes → items · quantities → packing list → PO
 - FX rates → bank advice · duty → declaration
- ↓ auto-populated Import Shipment (FR-523) with:
 - every landed cost line, its currency, its FX rate,
 - its supplier, its allocation basis
- ↓ human reviews a pre-filled allocation screen
- ↓ posts → cost layers updated → per-unit landed cost known

A day of clerical work becomes ten minutes of review. Nobody else can do this, because doing it requires an advanced landed cost engine (FR-524) on the other side of the extraction. Extraction without the engine produces a nice-looking form and no accounting.

The Document Set

DOCUMENT	EXTRACTS TO	ENABLED BY
Vendor bill (Arabic, thermal, creased)	Bill lines, auto three-way matched	FR-521 tolerances
Customs declaration + clearance invoice	Landed cost lines	FR-524
Packing list	Serial numbers, batch numbers, expiry dates	FR-342, FR-351
Bill of lading	Shipment tracking, in-transit dates	FR-523
Bank statement	Reconciliation candidates	FR-592
Carrier settlement file (PDF variants)	COD reconciliation	FR-1402
Cheque images	Post-dated cheque register	FR-594
WhatsApp order screenshot	Draft sales order + Trust lookup	FR-401, FR-600
Handwritten delivery note	Goods receipt draft	FR-501

Why It Gets Better Over Time

The learning loop (v1.0 AI-208): every human correction is stored as (supplier, document_type, field, extracted, corrected). Per-supplier layout hints mean the fiftieth bill from the same supplier extracts perfectly. **This compounds per tenant and cannot be copied — a competitor starting later starts at zero corrections.**

And the three-way match automation is where it becomes invisible: OCR the bill → auto-match against PO and goods receipt → **within tolerance, it posts without human involvement at all.** Only exceptions reach a person. That is the difference between "AI helps with data entry" and "data entry stopped existing."

I.3 Predictive Intelligence

The Reframe: Cash, Not Demand

Every ERP vendor demos demand forecasting. For an Egyptian SMB living through currency volatility, **cash forecasting is worth an order of magnitude more** — and it is far harder to do well, because it requires structured data from six different subsystems that most ERPs never connect.

You will have all six:

```
CASH FORECAST — next 60 days
  AR aging with per-customer payment behaviour      (FR-582)
+ Cash-with-Carrier receivables and carrier cycles  (FR-1401, 1405)
+ Post-dated cheques by maturity                    (FR-594)
- AP schedule with supplier terms                   (FR-505)
- LC and import commitments by stage                 (FR-523)
- Recurring costs (rent, salaries, utilities)        (FR-564)
± FX exposure on foreign-currency positions          (FR-595)
± Expected reorder outflows                          (FR-362)
-----
Daily projected balance, with confidence bands
and a flagged shortfall date
```

إنت هتقع في عجز 41,000 ج.م يوم 23. لو أخرت دفعة مورد الزجاج 5 أيام، أو حصلت 60% من متأخرات العملاء فوق 60 يوم، "تعدّي الأزمة."

That single feature justifies the entire subscription for a business owner, and it is unbuildable without the invincible core.

Demand Forecasting — The Three Stages

STAGE	METHOD	DATA REQUIRED	TRIGGER
1	Reorder point heuristic — transparent, explainable, honest	Any	Day 1
2	Croston's method for intermittent demand (most SMB SKUs are intermittent — applying continuous-demand models to intermittent data produces confident nonsense)	90+ days	Per item, automatic
3	Cross-tenant seasonality priors	Network scale	Phase 8c

Stage 3 is the second network effect and it is badly underrated. A brand-new merchant selling phone accessories gets Ramadan, Eid, back-to-school, and Black Friday seasonality curves on day one — derived from 5,000 merchants who came before, anonymized and aggregated at category level. Their competitor on Odoo needs three years of their own history to learn the same thing.

The Uniquely-Ours Layer: Net Demand

Because the Trust Network tracks returns, forecasting can predict **net** demand rather than gross:

"Forecast 340 units gross. Expected returns for this category and customer mix: 22%. Plan for 265 net. Two customers in your recent order flow have network return rates above 60% — excluding them changes the picture by 40 units."

No competitor can compute this. Not because their models are worse, but because they do not have cross-merchant return data and never will.

Reorder Optimization

With landed cost, lead time, FX exposure, MOQ, supplier terms, and carrying cost all structured, reordering stops being a threshold check and becomes a genuine optimization: *what to order, from whom, in what quantity, when, in which currency* — subject to a working capital constraint the merchant sets.

I.4 Anomaly Detection & Fraud Intelligence

The Substrate Advantage

Fraud detection is a signal-density problem. The core produces signals that most ERPs simply do not record:

- **Hash-chained journal entries (FR-561)** — tamper evidence is provable, not assumed
- **Immutable approval trail (FR-1211)** — who approved what, when, from where, on which device
- **Four-eyes records (FR-1217)** — including supplier bank detail changes
- **Cost layer lineage** — a margin anomaly traces to a specific layer, a specific receipt, a specific shipment
- **Serial genealogy (FR-344)** — every unit's full history
- **Cross-branch dimensional postings** — patterns visible across locations
- **Trust Network graph** — customer ↔ merchant ↔ outcome edges

Detection Tiers

Tier 1 — Rules (already live at launch). The V1 validation rules keep running. Every human dismissal becomes a labeled training example. By Phase 8 you have 18 months of labeled anomalies across thousands of tenants — **a training set no competitor can assemble.**

Tier 2 — Learned per-tenant baselines. Instead of "flag purchases 20% above average," the system learns *this* merchant's normal: their seasonal price patterns, their supplier variance, their staff's working hours, their typical discount behaviour. Alert volume drops, precision rises, alert fatigue (AI-412) stops being the binding constraint.

Tier 3 — Fraud patterns. Multi-signal detection of things a rule cannot express:

PATTERN	SIGNALS COMBINED
Ghost vendor	New supplier + bank detail set at creation + round amounts + single approver + no PO + no goods receipt
Kickback scheme	One buyer consistently uses one supplier at above-network prices, with tolerance overrides
Inventory shrinkage ring	Adjustment patterns correlated to specific staff, shifts, warehouses, and stock take variances
Return fraud	Serial returned that was never sold, or sold to a different customer (FR-345 catches the simple case; ML catches the organized one)
Approval circumvention	Documents split just below threshold — three POs at EGP 49,000 when the rule triggers at 50,000. This one is nearly universal and nearly never caught
COD collusion	Carrier-side patterns where "returns" cluster suspiciously by driver or route

Tier 4 — Graph-based network fraud. With the Trust Network exported to a graph store (FR-676), fraud *rings* become visible: clusters of phone numbers sharing addresses, ordering patterns, and return behaviour across many merchants simultaneously. A single merchant sees one bad customer. The network sees an organized operation.

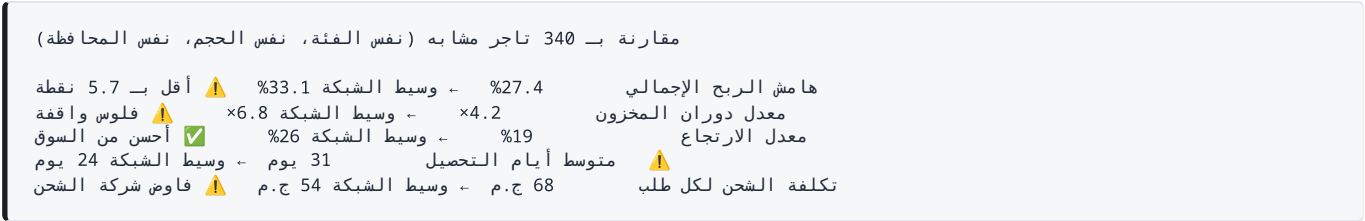
"شبكة احتيال محتملة: 14 رقم مختلف، نفس منطقة التسليم، 89% معدل ارتجاع، ضربوا 23 تاجر آخر آخر 60 يوم. 3 منهم عندك طلبات مفتوحة دلوقتي."

This is the capability that turns the Trust Network from a useful badge into critical infrastructure — and it is the point at which the product becomes genuinely difficult to leave.

I.5 The Network Intelligence Layer

The retention feature, available only at scale, and structurally impossible for self-hosted competitors (Odoo) or siloed ones (NetSuite).

Anonymized, aggregated, k-anonymous benchmarking:



Every line here is an actionable, quantified, credible reason to keep paying. And the same privacy architecture already built for the Trust Network — HMAC hashing, k-anonymity thresholds, consent, reciprocity — applies directly. The infrastructure exists; this is a reporting layer on top of it.

I.6 Why Legacy Cannot Follow

COMPETITOR	STRUCTURAL BLOCKER
Odoo	Self-hosted and fragmented. No cross-customer data by design. Their AI will only ever see one company's books. No Trust Network, ever
NetSuite / SAP	Multi-tenant but contractually siloed; enterprise customers will not permit cross-tenant learning. Also: no Egyptian COD reality, no ETA depth
Zoho	Could build cross-tenant intelligence, but has no Egyptian density and no COD/carrier settlement data. Would need to win the market first — and the market is what produces the data
Daftra / Qoyod / Wafeq	Architecturally static. No cost layer ledger, no dimensional postings, no immutable movement ledger. Their AI would be reasoning over mush. This is the moat, and they cannot cross it without rebuilding their core
A well-funded new entrant	Can copy the architecture. Cannot copy 18 months of network data, per-supplier OCR corrections, or labeled anomaly outcomes. Time is the moat

The one-sentence version: Anyone can add AI. Almost nobody can add the fourteen months underneath it, and by the time they start, the network data has compounded for two years.

I.7 Phase 8 Sequencing

Sub-Phase	Content	Duration	Prerequisite
8a — Foundations	On-soil GPU procurement · vLLM/Ollama deployment · tier router with data-classification enforcement · Copilot Levels 1–2 over the existing reporting service layer · golden-question suite	5 wks	FR-1501 complete (it will be, from Phase 1)
8b — Document Intelligence	VLM pipeline · vendor bills · customs/landed cost extraction · packing list serial capture · learning loop · automated three-way match	6 wks	8a
8c — Predictive & Proactive	Cash forecasting · demand forecasting stages 2–3 · reorder optimization · morning brief · Copilot Levels 3–4	6 wks	8b, ~1,000 tenants for stage 3
8d — Fraud & Network Intelligence	ML anomaly tiers 2–3 · graph export and ring detection · benchmarking layer	6 wks	8c, network density

Total ~23 weeks, but note it is *post-launch, revenue-funded, and informed by real merchants* — which is exactly the sequencing Appendix A of v1.0 was arguing for, and which you have now arrived at from the other direction.

Critical dependency to start now, not in Phase 8: GPU availability from Egyptian providers is scarce and lead times are long. Open that conversation with your hosting provider during Phase 1.

I.8 Non-Negotiable Guardrails

Carried forward from v1.0 §7.7, restated because they matter more as capability grows:

1. **AI never writes to the ledger directly.** It drafts; the approval engine governs; humans commit.
2. **The AI is never a valid approver identity.** Enforced at the database level, not in application logic.
3. **Row-level tenant data never leaves the on-soil infrastructure.** Enforced by a data-classification tag checked at the router boundary, with a unit test.
4. **Every AI-influenced record carries provenance:** model version, prompt hash, confidence, and reviewing human.
5. **Every AI answer drills through to its source transactions.** No exceptions.
6. **Kill switch per feature, per tenant, and platform-wide.**
7. **A model upgrade must pass the golden-question suite before rollout.** Copilot accuracy is a regression-testable property; treat it like one.

BLUEPRINT II — Phase 0: The Refactoring Execution Plan

Duration: 7 weeks · **Goal:** transform the existing repository into the foundation the next 53 weeks stand on, without a rewrite.

II.0 Principles of This Refactor

1. **Interfaces stay; implementations are replaced.** `event_bus.publish()` keeps its exact signature. Its guts change completely. Nothing that calls it is touched.

2. **The database is greenfield.** Zero production tenants (A-16, confirmed). We drop everything and write one clean baseline migration. **This is an enormous gift — do not waste it by preserving a schema you will fight for three years.**
3. **Fix seams in dependency order.** Session management → events → workers → data model. Every step depends on the one before it. Doing them out of order means doing them twice.
4. **No feature work during Phase 0.** None. The temptation to "just add invoicing while we're in here" is how Phase 0 becomes Phase 0 through 3.
5. **Every step ends green.** The repo must build, run, and pass tests at the end of each step. No four-week broken-branch death marches.
6. **Tests before the thing they test,** specifically for tenancy isolation and outbox delivery. These are the two properties where a bug is silent and catastrophic.

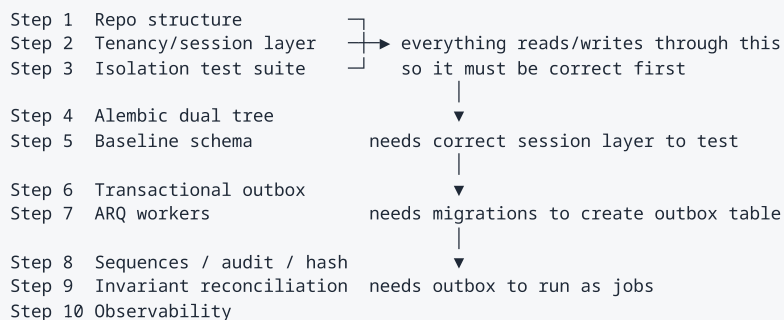
II.1 Repository Triage

Exactly what happens to each existing piece:

PATH	VERDICT	ACTION
<code>docker-compose.yml</code>	EXTEND	Add PgBouncer, ARQ worker, MinIO, Prometheus/Grafana/Loki
<code>app/core/config.py</code>	KEEP	Add settings for ARQ, outbox, PgBouncer, secrets refs
<code>app/core/database.py</code>	REPLACE	The <code>SET search_path</code> approach is incompatible with pooling. Full rewrite of ~150 lines. §II.3 Step 2
<code>app/core/security.py</code>	KEEP	JWT + bcrypt + Redis refresh is correct. Extend for branch scoping later
<code>app/core/event_bus.py</code>	REPLACE INTERNALS	Public interface preserved. <code>asyncio.create_task</code> → outbox write. §II.3 Step 6
<code>app/modules/system/models.py</code>	EXTEND	Tenant gains <code>shard_id</code> , <code>db_cluster_ref</code> , <code>schema_version</code> , <code>plan</code> , <code>trust_consent</code> , <code>provisioning_state</code>
<code>app/modules/system/router.py</code>	REFACTOR	The 7-step inline provisioning pipeline becomes a resumable job (FR-102). Route returns 202
<code>app/modules/accounting/models.py</code>	EXTEND HEAVILY	Hierarchy, dimensions, hash chain, control accounts, periods
<code>app/modules/accounting/events.py</code>	REFACTOR	Becomes an idempotent consumer registered against the outbox relay
<code>app/plugins/inventory/router.py</code>	DELETE	The <code>/invoices/simulate</code> route served its purpose proving the event bus. It is now misleading. Delete it
<code>app/plugins/inventory/models.py</code>	REPLACE	<code>items</code> with mutable <code>price</code> / <code>cost</code> is architecturally wrong. Rebuilt around the cost layer ledger
<code>app/modules/trust/*</code>	KEEP + HARDEN	Logic is sound. Add HMAC pepper, k-anonymity, reciprocity, consent in Phase 5
<code>alembic/</code>	RESTRUCTURE	Single tree → dual tree (<code>public</code> / <code>tenant</code>). §II.3 Step 4
<code>app/main.py</code>	REFACTOR	Router registration becomes manifest-driven
Frontend, all of it	KEEP	App Router, route groups, dynamic sidebar, lazy loading, Error Boundaries, Zustand, React Query — all correct. Phase 0 frontend work is the design system, in parallel. §II.4

Roughly 85% of the repository survives Phase 0 untouched. The 15% that changes is load-bearing.

II.2 Why This Exact Sequence



Attempting the outbox before fixing session management means writing outbox code against a broken session model and rewriting it. Attempting the data model before Alembic restructuring means a baseline migration you cannot apply per-tenant.

II.3 Step-by-Step Execution

STEP 0 — Preparation (½ day)

```
git tag v0-pre-refactor && git push --tags
git checkout -b phase-0/foundation
```

Freeze all feature work. Snapshot the current OpenAPI spec (`/openapi.json`) — it is your regression reference for what must keep working. Write down the handful of end-to-end flows that work today (register → login → create contact → simulate invoice → see journal entry); these become the smoke tests that must stay green through every step.

STEP 1 — Repository Restructure (2 days)

Move from a flat module layout to explicit layers. Pure file movement plus import fixes — zero logic change.

```

backend/app/
├── core/
│   ├── config.py
│   ├── db/
│   │   ├── engine.py          # engine + pool construction
│   │   ├── session.py        # tenant-aware session factory ← Step 2
│   │   ├── base.py           # declarative base, mixins
│   │   └── context.py         # ContextVars: tenant, user, session
│   ├── events/
│   │   ├── bus.py            # public interface (unchanged signature)
│   │   ├── outbox.py         # outbox model + emit ← Step 6
│   │   ├── relay.py          # relay worker ← Step 6
│   │   ├── consumers.py      # registry + idempotency ← Step 6
│   │   └── schemas.py        # DomainEvent, versioned payloads
│   ├── tenancy/
│   │   ├── middleware.py      # X-Tenant-ID → context
│   │   ├── provisioning.py    # resumable state machine ← Step 5
│   │   └── migrations.py     # orchestrator ← Step 4
│   ├── security/
│   ├── audit/                # ← Step 8
│   ├── sequences/            # ← Step 8
│   └── observability/         # ← Step 10
├── modules/                  # Core domain (accounting, contacts, system, trust)
├── plugins/                  # Optional domain (inventory, shipping, ...)
├── workers/                  # ARQ definitions ← Step 7
│   ├── settings.py
│   ├── queues.py
│   └── tasks/
└── main.py

```

Exit: app boots, existing smoke tests pass, no behavioural change.

STEP 2 — The Tenancy & Session Layer (4 days) ⚠️ *Highest-risk step in Phase 0*

The problem. Your current model sets `SET search_path TO tenant_x` on the session. PgBouncer in transaction mode returns the connection to the pool after each transaction, but **search_path is session state and persists on that physical connection**. The next request to grab that connection inherits the previous tenant's search path. Without PgBouncer you exhaust connections; with PgBouncer you serve cross-tenant data. Both outcomes are unacceptable, and the second is fatal.

The fix: `schema_translate_map`. Models declare a symbolic schema; SQLAlchemy substitutes the real schema name **at SQL compilation time**, so it is embedded in the statement rather than held as connection state. Fully pooling-safe.

```

# core/db/base.py
from sqlalchemy.orm import DeclarativeBase

class PublicBase(DeclarativeBase):
    """Tables in the public schema: tenants, plugins, global_reputation, outbox."""
    __table_args__ = {"schema": "public"}

class TenantBase(DeclarativeBase):
    """Tables inside each tenant schema. 'tenant' is a SYMBOL, never a real schema."""
    __table_args__ = {"schema": "tenant"}

```

```

# core/db/context.py
from contextvars import ContextVar
from uuid import UUID
from sqlalchemy.ext.asyncio import AsyncSession

current_tenant_id: ContextVar[UUID | None] = ContextVar("current_tenant_id", default=None)
current_user_id: ContextVar[UUID | None] = ContextVar("current_user_id", default=None)
current_session: ContextVar[AsyncSession | None] = ContextVar("current_session", default=None)

def schema_for(tenant_id: UUID) -> str:
    return f"tenant_{tenant_id.hex}"

```

```
# core/db/session.py
from contextlib import asynccontextmanager
from uuid import UUID
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker
from app.core.db.engine import engine
from app.core.db.context import schema_for, current_session

_factory = async_sessionmaker(engine, expire_on_commit=False, class_=AsyncSession)

@asynccontextmanager
async def tenant_session(tenant_id: UUID):
    """
    Sole entry point for tenant-scoped DB access.
    Safe under PgBouncer transaction pooling: no session-level state is set.
    """
    async with _factory() as session:
        await session.connection(
            execution_options={"schema_translate_map": {"tenant": schema_for(tenant_id)}}
        )
        token = current_session.set(session)
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
        finally:
            current_session.reset(token)

@asynccontextmanager
async def public_session():
    async with _factory() as session:
        token = current_session.set(session)
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
        finally:
            current_session.reset(token)
```

Four rules that must be enforced, not merely documented:

RULE	ENFORCEMENT
No <code>SET search_path</code> anywhere in the codebase	CI grep gate: <pre>grep -rn "search_path" app/ && exit 1</pre>
No raw <code>text()</code> SQL without explicit schema qualification — <code>schema_translate_map</code> does not rewrite raw SQL strings	CI lint rule flagging <code>text(</code> outside an allowlist
No direct use of <code>_factory</code> ; all access via <code>tenant_session</code> / <code>public_session</code>	Module-private naming + review
Every ARQ job establishes its own tenant context — never inherits	Job wrapper enforces it (Step 7)

PgBouncer configuration:

```
[databases]
omni_erp = host=postgres port=5432 dbname=omni_erp

[pgbouncer]
pool_mode = transaction
max_client_conn = 2000
default_pool_size = 40
server_reset_query =          # deliberately empty: we set no session state
```

And in SQLAlchemy, because PgBouncer transaction mode is incompatible with server-side prepared statements:

```
engine = create_async_engine(
    settings.DATABASE_URL,
    poolclass=NullPool,                    # PgBouncer owns pooling
    connect_args={"statement_cache_size": 0,
                  "prepared_statement_cache_size": 0}, # required for asyncpg + PgBouncer
)
```

Exit: all reads and writes flow through the new session layer. PgBouncer live in docker-compose. Step 3 proves it.

STEP 3 — The Isolation Test Suite (3 days)

This suite outlives every other artifact in Phase 0. It is the test that stops the company-ending bug.

```
# tests/isolation/conftest.py
@pytest.fixture
async def two_tenants():
    """Provision tenant A and tenant B with deliberately identical-looking data."""
    a = await provision_test_tenant("alpha")
    b = await provision_test_tenant("beta")
    await seed(a, contact_name="Ahmed", invoice_total=1000)
    await seed(b, contact_name="Ahmed", invoice_total=9999)
    return a, b
```

Every access path gets a test. The bug always hides in the path nobody thought of:

```
async def test_api_cannot_read_other_tenant(two_tenants): ...
async def test_pooled_connection_reuse_does_not_leak(two_tenants):
    """
    THE critical test. Hammer both tenants concurrently through a pool
    smaller than the concurrency, forcing connection reuse, and assert
    every response belongs to the requesting tenant.
    """
    results = await asyncio.gather(*[
        fetch_invoices(a if i % 2 == 0 else b) for i in range(200)
    ])
    for tenant, rows in results:
        assert all(r.tenant_marker == tenant.id for r in rows)

async def test_worker_job_cannot_leak(two_tenants): ...
async def test_cache_key_namespacing(two_tenants): ...
async def test_websocket_channel_scoping(two_tenants): ...
async def test_report_export_scoping(two_tenants): ...
async def test_event_consumer_writes_to_correct_schema(two_tenants): ...
async def test_trust_network_returns_no_tenant_identity(two_tenants): ...
async def test_error_messages_leak_no_other_tenant_data(two_tenants): ...
```

Add to CI as a required, non-skippable check. When branches arrive in Phase 1c, this suite gains a parallel cross-branch dimension (FR-1304) — same fixtures, same shape.

Exit: suite green, wired into CI as blocking.

STEP 4 — Alembic Dual Tree + Migration Orchestrator (4 days)

```
alembic/
├── public/           # global tables. Runs once per DB cluster.
│   ├── env.py       # target_metadata = PublicBase.metadata
│   └── versions/
└── tenant/          # per-schema tables. Runs N times, once per tenant.
    ├── env.py       # target_metadata = TenantBase.metadata
    └── versions/
```

`alembic/tenant/env.py` — the key detail is `include_schemas=False` plus a translate map, so one migration script applies cleanly to any schema:

```
def run_migrations_online():
    schema = context.get_x_argument(as_dictionary=True)["schema"]
    connectable = engine.execution_options(schema_translate_map={"tenant": schema})
    with connectable.connect() as connection:
        context.configure(
            connection=connection,
            target_metadata=TenantBase.metadata,
            version_table_schema=schema,      # each schema tracks its own version
            include_schemas=False,
        )
    with context.begin_transaction():
        context.run_migrations()
```

The orchestrator (ARCH-701) — build it now, at tenant #0, when it costs 4 days. At tenant #3,000 it costs a weekend of downtime and your reputation.

```
# core/tenancy/migrations.py
class TenantMigrationOrchestrator:
    """
    Runs tenant migrations as batched background jobs with per-tenant state.
    NEVER a single blocking `alembic upgrade head` across the fleet.
    """
    async def plan(self, target_revision: str) -> MigrationPlan:
        """Which tenants are behind? Group by current revision."""

    async def execute(self, plan: MigrationPlan, concurrency: int = 20):
        """
        Dispatch one ARQ job per tenant, `concurrency` at a time.
        Each job: mark running → migrate → verify → mark done | mark failed.
        Resumable: a crash mid-fleet resumes from tenant state, not from zero.
        """

    async def status(self) -> FleetStatus:
        """pending / running / done / failed counts + failure detail."""
```

Backing table in `public`:

```
CREATE TABLE public.tenant_migrations (
    tenant_id      uuid NOT NULL,
    from_revision  text,
    to_revision    text NOT NULL,
    state          text NOT NULL CHECK (state IN ('pending','running','done','failed')),
    attempts       int  NOT NULL DEFAULT 0,
    error          text,
    started_at     timestampz,
    finished_at    timestampz,
    PRIMARY KEY (tenant_id, to_revision)
);
```

Migration discipline, adopted from day one (ARCH-702/703): - Expand → migrate → contract. Never a breaking change in a single deploy. - `CREATE INDEX CONCURRENTLY` always. - `ADD COLUMN` without a volatile default (no table rewrite). - Constraints added `NOT VALID`, then `VALIDATE CONSTRAINT` separately. - **CI rejects any migration containing `DROP COLUMN`, `ALTER COLUMN TYPE`, or a non-concurrent index creation** unless the PR carries an `expand-contract-approved` label.

Exit: `omni migrate --tenants --to head --concurrency 20` works. Tested against 50 seeded tenants; kill it mid-run and confirm it resumes correctly.

STEP 5 — Wipe and Rebuild the Schema (5 days)

```
docker compose down -v      # the gift of zero production tenants
```

Write **one baseline migration per tree**. Not the full 60-table target model — Phase 0's baseline covers foundations plus enough to keep existing flows alive:

```
public baseline: tenants (with shard_id, db_cluster_ref, schema_version, plan, provisioning_state, trust_consent) ·
users · tenant_provisioning_jobs · tenant_migrations · plugins · tenant_plugins · outbox_events · platform_audit_log ·
global_reputation
```

```
tenant baseline: users · contacts (with phone_e164) · accounts (hierarchical, with is_control, is_system) ·
accounting_periods · journal_entries (with entry_hash, prev_hash, source_event_id) · journal_lines (with branch_id,
cost_center_id, contact_id, currency + base amounts) · branches · document_sequences · audit_logs · processed_events ·
attachments
```

Two decisions to make now, because they are painful later:

(a) Every table gets the standard mixin.

```
class BaseMixin:
    id = mapped_column(UUID, primary_key=True, default=uuid7)
    created_at = mapped_column(TIMESTAMP(timezone=True), server_default=func.now())
    updated_at = mapped_column(TIMESTAMP(timezone=True), onupdate=func.now())
    created_by = mapped_column(UUID, nullable=True)
    updated_by = mapped_column(UUID, nullable=True)
    deleted_at = mapped_column(TIMESTAMP(timezone=True), nullable=True)
```

Use **UUIDv7**, not v4 — time-ordered UUIDs dramatically reduce B-tree index fragmentation on high-insert tables like `stock_movements`, which will be your largest table by an order of magnitude.

(b) `branch_id` goes on every transactional table and every journal line NOW, nullable, defaulted to a "Main" branch created at provisioning. Multi-branch features arrive in Phase 1c, but **retrofitting the column later means touching every table, every query, and every index**. The column costs nothing today.

Provisioning becomes resumable (FR-102):

```
class ProvisioningState(str, Enum):
    CREATED = "created"; SCHEMA_CREATED = "schema_created"
    TABLES_CREATED = "tables_created"; COA_SEEDED = "coa_seeded"
    BRANCH_CREATED = "branch_created"; TREASURY_CREATED = "treasury_created"
    WAREHOUSE_CREATED = "warehouse_created"; SEQUENCES_CREATED = "sequences_created"
    PERIODS_CREATED = "periods_created"; COMPLETE = "complete"
```

`POST /auth/register` writes the tenant row and enqueues the job, returning **202** with a poll URL. Each step is idempotent and advances the state. A reconciliation worker sweeps stalled jobs. **A failure at step 6 of 10 never leaves a half-built schema again.**

Exit: `docker compose up` from clean → migrations run → register a tenant → provisioning completes → smoke tests green.

STEP 6 — The Transactional Outbox (5 days) ⚠️ *The step that fixes the silent corruption*

Design refinement to ARCH-301: v2.0 placed `outbox_events` in the tenant schema. **Put it in public with a `tenant_id` column instead.** The same-transaction guarantee is a property of the *transaction*, not the schema — one connection can write a tenant-schema invoice and a public-schema outbox row atomically. The benefit is decisive: the relay worker runs **one** query across all tenants instead of iterating thousands of schemas. Outbox rows are transient, so they don't belong in per-tenant backups anyway.

```
CREATE TABLE public.outbox_events (
    id            uuid PRIMARY KEY,
    tenant_id     uuid NOT NULL,
    aggregate_type text NOT NULL,
    aggregate_id   uuid NOT NULL,
    event_type     text NOT NULL,
    event_version  int  NOT NULL DEFAULT 1,
    payload        jsonb NOT NULL,
    occurred_at    timestamptz NOT NULL DEFAULT now(),
    published_at   timestamptz,
    attempts       int NOT NULL DEFAULT 0,
    last_error     text
);
-- partial index: the relay only ever scans unpublished rows
CREATE INDEX CONCURRENTLY ix_outbox_unpublished
ON public.outbox_events (occurred_at)
WHERE published_at IS NULL;
```

The zero-call-site-change trick. Every existing `await event_bus.publish(DomainEvent(...))` call keeps working unmodified, because `publish` reads the ambient session from the `ContextVar` set in Step 2:

```
# core/events/bus.py - PUBLIC INTERFACE UNCHANGED
class EventBus:
    async def publish(self, event: DomainEvent, session: AsyncSession | None = None) -> None:
        """
        Was: asyncio.create_task(dispatch(event))      ← lost on crash
        Now: INSERT into outbox within the caller's transaction.
        Signature preserved: every existing call site compiles untouched.
        """
        session = session or current_session.get()
        if session is None:
            raise RuntimeError("publish() requires an active DB session")
        session.add(OutboxEvent(
            id=uuid7(),
            tenant_id=event.tenant_id,
            aggregate_type=event.aggregate_type,
            aggregate_id=event.aggregate_id,
            event_type=event.event_type,
            event_version=event.version,
            payload=event.payload,
        ))
        # No commit here. The caller's transaction commits both, or neither.

event_bus = EventBus()
```

That is the whole answer to "how do we not rewrite the repo." One file changes; the fifty call sites do not.

The relay worker:

```
# core/events/relay.py
BATCH = 200

async def relay_tick(redis: Redis):
    async with public_session() as s:
        rows = (await s.execute(
            select(OutboxEvent)
            .where(OutboxEvent.published_at.is_(None))
            .order_by(OutboxEvent.occurred_at)
            .limit(BATCH)
            .with_for_update(skip_locked=True)      # safe with N relay replicas
        )).scalars().all()

        for e in rows:
            # Partition by aggregate_id to guarantee per-aggregate ordering (ARCH-306)
            stream = f"events:{e.event_type}:{hash(e.aggregate_id) % STREAM_PARTITIONS}"
            await redis.xadd(stream, {"payload": serialize(e)}, maxlen=1_000_000)
            e.published_at = func.now()
```

Run it as an ARQ cron at ~200ms, and additionally trigger it via `LISTEN/NOTIFY` on insert for low latency. Poll alone is a correct, simple fallback; `NOTIFY` is the optimization.

Idempotent consumers — the accounting handler is refactored, not rewritten:

```
# core/events/consumers.py
def consumer(event_type: str, name: str):
    def deco(fn):
        async def wrapped(event: DomainEvent):
            async with tenant_session(event.tenant_id) as s:
                # Unique (consumer_name, event_id) makes replay a no-op
                already = await s.execute(
                    insert(ProcessedEvent)
                    .values(consumer_name=name, event_id=event.id)
                    .on_conflict_do_nothing()
                    .returning(ProcessedEvent.event_id)
                )
                if already.scalar() is None:
                    return # already handled – exactly-once effect
                await fn(event, s) # same transaction as the marker
            registry.register(event_type, wrapped)
        return wrapped
    return deco

@consumer("invoice.created", name="accounting.post_invoice")
async def post_invoice_journal(event: DomainEvent, s: AsyncSession):
    ... # your existing logic, essentially unchanged
```

Dead-letter queue after N attempts, with alerting and a replay UI for you (ARCH-304).

Test that must pass before this step is done:

```
async def test_crash_between_commit_and_dispatch_loses_nothing():
    """Kill the relay mid-batch; restart; every event still delivered exactly once."""
```

Exit: `asyncio.create_task` deleted from the event path. Chaos test green. The R-02 risk is closed.

STEP 7 — ARQ Workers (3 days)

```
# workers/settings.py
class WorkerSettings:
    redis_settings = RedisSettings.from_dsn(settings.REDIS_URL)
    functions = [relay_tick, migrate_tenant, run_invariants,
                 provision_tenant, reconcile_provisioning]
    cron_jobs = [
        cron(relay_tick, second={0,1,...}), # effectively continuous
        cron(run_invariants, hour=2, minute=0),
        cron(verify_backup, hour=3, minute=0),
        cron(reconcile_provisioning, minute={0,15,30,45}),
    ]
    max_jobs = 20
    job_timeout = 600
```

Queue tiering (ARCH-401) — four separate worker pools so a 10,000-row import never delays an ETA submission:

QUEUE	WORKERS	CONTENTS
critical	4	ETA submission, payment webhooks, carrier webhooks
default	8	Postings, notifications, cache warming
bulk	4	Imports, exports, report generation, cost recalculation
scheduled	2	Cron: invariants, backup verification, FX fetch

Every job establishes its own tenant context — never inherits (ARCH-403):

```
def tenant_job(fn):
    @functools.wraps(fn)
    async def wrapped(ctx, tenant_id: UUID, *a, **kw):
        token = current_tenant_id.set(tenant_id)
        try:
            async with tenant_session(tenant_id) as s:
                return await fn(ctx, s, *a, **kw)
        finally:
            current_tenant_id.reset(token)
    return wrapped
```

Tenant fairness (ARCH-402): a per-tenant concurrency semaphore in Redis so one large tenant cannot monopolize a pool.

Exit: worker service in docker-compose, queue-depth metrics exported, isolation suite extended to cover worker paths.

STEP 8 — Sequences, Audit Log, Hash Chain (4 days)

Document sequences (FR-108) — gapless and concurrency-safe. `MAX(number)+1` is a race condition, not an implementation.

```
CREATE TABLE tenant.document_sequences (
    id          uuid PRIMARY KEY,
    doc_type    text NOT NULL,
    branch_id   uuid,
    fiscal_year int NOT NULL,
    prefix      text NOT NULL,
    padding     int NOT NULL DEFAULT 5,
    next_value  bigint NOT NULL DEFAULT 1,
    UNIQUE (doc_type, branch_id, fiscal_year)
);
```

Allocation uses `SELECT ... FOR UPDATE` on the sequence row inside the document's own transaction — so a rolled-back document does not consume a number, which is what "gapless" actually requires.

Audit log (FR-107) — append-only, with a DB-level guard:

```
CREATE RULE audit_no_update AS ON UPDATE TO tenant.audit_logs DO INSTEAD NOTHING;
CREATE RULE audit_no_delete AS ON DELETE TO tenant.audit_logs DO INSTEAD NOTHING;
```

Captures actor, timestamp, IP, user agent, before/after JSONB diff, and `source ∈ {ui, api, ai_assisted, system_event, import}`.

Hash-chained journal entries (FR-561) — the trust argument, and it costs a day:

```
async def compute_entry_hash(s, entry) -> tuple[str, str]:
    # Serialize on the chain head to prevent concurrent forks
    prev = (await s.execute(
        select(JournalEntry.entry_hash)
        .where(JournalEntry.period_id == entry.period_id)
        .order_by(JournalEntry.sequence_no.desc())
        .limit(1).with_for_update()
    )).scalar() or GENESIS_HASH
    payload = canonical_json({
        "date": entry.date, "lines": [
            {"account": l.account_code, "debit": str(l.debit),
             "credit": str(l.credit), "dims": l.dimensions}
            for l in sorted(entry.lines, key=lambda x: x.line_no)
        ],
    })
    return prev, sha256((prev + payload).encode()).hexdigest()
```

A nightly job walks each period's chain and verifies it. **A break is a P1 page.**

Exit: concurrency test — 500 simultaneous document creations produce 500 unique gapless numbers and an unbroken hash chain.

STEP 9 — Invariant Reconciliation (2 days)

The smoke alarm for the ledger (ARCH-308). Nightly per tenant, plus on demand:

INVARIANT	FAILURE SEVERITY
Every journal entry: $\Sigma \text{ debits} = \Sigma \text{ credits}$	P1 page
Every posted invoice has exactly one balanced journal entry	P1 page
Hash chain unbroken for every open period	P1 page
$\Sigma \text{ stock_movements.qty} = \text{stock_levels.qty_on_hand}$ per item/warehouse	P1 page (from Phase 1a)
Stock valuation = Inventory GL account balance	P1 page (Phase 1a)
AR subledger total = AR control account balance	P1 page
AP subledger total = AP control account balance	P1 page
Inter-branch clearing nets to zero across branches	P1 page (Phase 1c)
Outbox backlog < threshold and oldest unpublished < 5 min	P2 alert
No unreconciled provisional costs in a closing period	Blocks close (Phase 1a)

Results land in a `reconciliation_runs` table with per-check status and detail, surfaced in an internal ops dashboard. **Write the framework now with the checks that apply today; each later phase adds its own checks to the existing harness.**

STEP 10 — Observability (3 days)

```
# structured logging – every line carries tenant_id, user_id, request_id, trace_id
structlog.configure(processors=[
    add_tenant_context,      # pulls from ContextVars
    structlog.processors.TimeStamper(fmt="iso"),
    structlog.processors.JSONRenderer(),
])
```

- **OpenTelemetry** tracing across API → outbox → relay → worker → DB. A single trace id must follow an invoice from HTTP request to journal entry.
- **Sentry/GlitchTip**, self-hosted on-soil, with tenant context and release tagging.
- **Prometheus metrics**: RED per endpoint, queue depth and age, outbox lag, DB pool saturation, cache hit rate, invariant failures.
- **Business alerts, not just infra**: invariant failure, outbox backlog, provisioning failure, hash chain break.

STEP 11 — Egyptian Infrastructure (parallel track, ~3 weeks)

Runs alongside Steps 1–10. **This is the ideal work package for a third engineer joining mid-Phase-0** — it is cleanly separable and touches almost no application code.

INF-101 Patroni + etcd HA · INF-102 pgBackRest WAL archiving to MinIO · **INF-103 nightly automated restore verification** · INF-104 off-site replication to a second Egyptian facility · INF-105 MinIO object storage · INF-106 Prometheus/Grafana/Loki · INF-107 WAF and TLS · INF-109 Vault · INF-110 runbooks · INF-111 staging parity · INF-112 datacentre due diligence.

INF-103 is the single highest-value item on this list. A backup that has never been restored is a belief, not a capability.

STEP 12 — CI Gates (2 days)

A PR merges only if **all** of these pass:

```
- isolation-suite           # blocking, never skippable
- outbox-chaos-test        # kill relay mid-batch, assert exactly-once
- concurrency-test         # 500 concurrent doc creations
- invariant-test           # seeded ledger passes all checks
- migration-safety-lint    # no DROP COLUMN / ALTER TYPE / non-concurrent index
- search-path-grep         # zero occurrences outside the allowlist
- raw-sql-lint             # text() only in the allowlist
- coverage                 # ≥80% overall, 100% on tenancy + posting + hashing
- bundle-size              # frontend budgets
- lighthouse-ci            # frontend performance budgets
```

II.4 Frontend Track (parallel, weeks 1–7)

Your co-developer does not wait for the backend. Phase 0 frontend work has almost no backend dependency:

WEEK	WORK
1–2	Design tokens (§8.2 of v1.0): typography, tabular figures, spacing, semantic colour, light/dark as a pure token swap
2–3	RTL-first component primitives. Logical properties only; add the ESLint rule banning <code>left / right</code> now , before there is a codebase to retrofit
3–5	The <code><DataTable/></code> per §8.5 — server-side everything, virtualization, filter builder, saved views, bulk selection. Used on ~30 screens; it deserves 2 weeks
5–6	Command palette shell (§K) — navigation and actions. AI slot left empty until Phase 8
6–7	Arabic PDF pipeline (UX-205). Prove Arabic shaping, bidi, and font embedding in a generated PDF <i>now</i> . This is the most common embarrassing failure in this market, and discovering it in Phase 4 is far worse than discovering it in week 6

II.5 Exit Criteria — Phase 0 Is Done When

1. ☒ Isolation suite green, including the pooled-connection-reuse test, in CI as blocking
2. ☒ `grep -r "search_path" app/` returns nothing outside the allowlist
3. ☒ Outbox chaos test green — relay killed mid-batch, every event delivered exactly once
4. ☒ `asyncio.create_task` no longer appears anywhere in the event path
5. ☒ Migration orchestrator migrates 50 seeded tenants, survives a mid-run kill, and resumes
6. ☒ Tenant provisioning is resumable — killed at step 6 of 10, it completes on retry
7. ☒ 500 concurrent document creations → 500 unique gapless numbers, unbroken hash chain
8. ☒ Nightly invariant job runs and pages on a deliberately corrupted entry
9. ☒ A trace id follows a request from HTTP through outbox, relay, worker, to journal entry
10. ☒ **A tenant schema has been restored from backup into a scratch instance and verified**
11. ☒ Postgres failover drilled, timed, and documented
12. ☒ Staging mirrors production topology including replication
13. ☒ Frontend design system published, DataTable feature-complete, Arabic PDF proven

14. All CI gates active and blocking

Do not begin Phase 1a until all fourteen are true. Every one of them is something that gets exponentially more expensive to fix later, and several of them are things you cannot fix at all once real merchants' money is in the system.

II.6 What NOT to Do in Phase 0

-  **No feature work.** Not one invoice field. This is the discipline that determines whether Phase 0 takes 7 weeks or 16.
-  **Do not build the full 60-table target model.** Baseline only. Inventory and costing tables belong to Phase 1a, designed with the costing engine, not ahead of it.
-  **Do not start ETA.** Except procurement conversations with signing providers, which should start week 1 and run in the background.
-  **Do not touch the Trust Network logic.** It works. It gets hardened in Phase 5.
-  **Do not optimize prematurely.** No read replica routing, no materialized views, no ClickHouse. Correctness first; those arrive when measurements demand them.
-  **Do not skip Step 3 to "get to the interesting work."** The isolation suite is the interesting work. It is the one thing standing between you and the failure mode that ends companies.

II.7 Week-by-Week

WEEK	BACKEND (YOU)	BACKEND (3RD ENG, IF HIRED)	FRONTEND (CO-DEV)
1	Steps 0–1, start Step 2	INF-112 DD, provider selection	Design tokens
2	Step 2 (session layer)	INF-101 Patroni HA	RTL primitives, ESLint rules
3	Step 3 (isolation suite)	INF-102 pgBackRest, INF-105 MinIO	DataTable core
4	Step 4 (Alembic + orchestrator)	INF-103 restore verification	DataTable filters + views
5	Step 5 (baseline schema + provisioning)	INF-106 observability stack	DataTable bulk actions, virtualization
6	Step 6 (outbox)	INF-104 off-site, INF-111 staging	Command palette
7	Steps 7–10 (ARQ, sequences, audit, invariants, o11y), Step 12	INF-110 runbooks, failover drill	Arabic PDF pipeline

Without a third engineer, Steps 11's infrastructure work extends Phase 0 to roughly **9 weeks**. That is the concrete, quantified case for the hire, and it recurs: infrastructure operations remain a permanent 15–20% tax on one engineer thereafter.